Dropout and Overfitting in a Character-Level Transformer Trained on Tiny Shakespeare

Samrath Singh Dhingra

July 2026

Abstract

I train a 10.65M-parameter GPT-style transformer to predict the next character of the Tiny Shakespeare corpus (1.1M characters) and ask how much its built-in dropout of 0.2 matters. Two runs of 2,000 iterations are compared that are identical in data, seed, architecture and optimiser, and differ only in dropout (0.2 or 0.0). With dropout the validation loss falls to 1.468 nats per character and is still improving slightly at the end, while the gap to the training loss grows to 0.41. Without dropout the model learns faster at first, reaches its best validation loss of 1.559 at iteration 1,000, and then overfits sharply: by iteration 2,000 the training loss is 0.40 and the validation loss has risen to 2.41. In text sampled from the best checkpoint of each run, 97.1% of generated words occur in the training text with dropout against 93.8% without. On a corpus this small, dropout is what keeps this model from memorising its training data. The findings rest on one seed per setting and loss evaluated only every 500 iterations.

1 Introduction

Transformer language models [1] are usually trained on far more text than they have parameters. The Tiny Shakespeare setting reverses that ratio: about ten parameters per training character. A model in this regime can lower its training loss by memorising, so regularisation should decide how well it generalises. Dropout [2] is the main regulariser in the standard nanoGPT character-level configuration [3]. This report first reproduces that configuration over a shortened schedule, then removes dropout and measures what changes.

2 Method

Data : The corpus is the Tiny Shakespeare file distributed with char-rnn [4]: 1,115,394 characters of plays in a single text file. Each character is one token, giving a vocabulary of 65 symbols. The first 90% of the file (1,003,854 tokens) is used for training and the final 10% (111,540 tokens) for validation. There is no separate test set.

Model : I use the nanoGPT implementation [3] of a decoder-only transformer: token and learned position embeddings, 6 blocks each containing causal self-attention with 6 heads and a $4\times$ GELU feed-forward layer, pre-layer normalisation, an embedding width of 384, a context of 256 characters, and an output layer that shares weights with the token embedding. The model has 10.65M parameters. Dropout, when enabled, is applied to the embeddings, the attention weights and the output of each residual branch.

Training : Both runs use AdamW [5] with $\beta = (0.9, 0.99)$, weight decay 0.1 on the weight matrices, and gradient clipping at norm 1.0. Each iteration draws 64 random windows of 256 characters from the training split, 16,384 tokens in total. The learning rate warms up linearly to 10^{-3} over 100 iterations and then follows a cosine decay towards 10^{-4} that is configured for 5,000 iterations. To fit a free Colab session I stop at 2,000 iterations, at which point the learning rate is still 7.05×10^{-4}. A run therefore sees 32.8M tokens, 32.6 times the size of the training split, and takes about 25 minutes on one NVIDIA T4 GPU (0.55 s per iteration plus 79 s per evaluation).

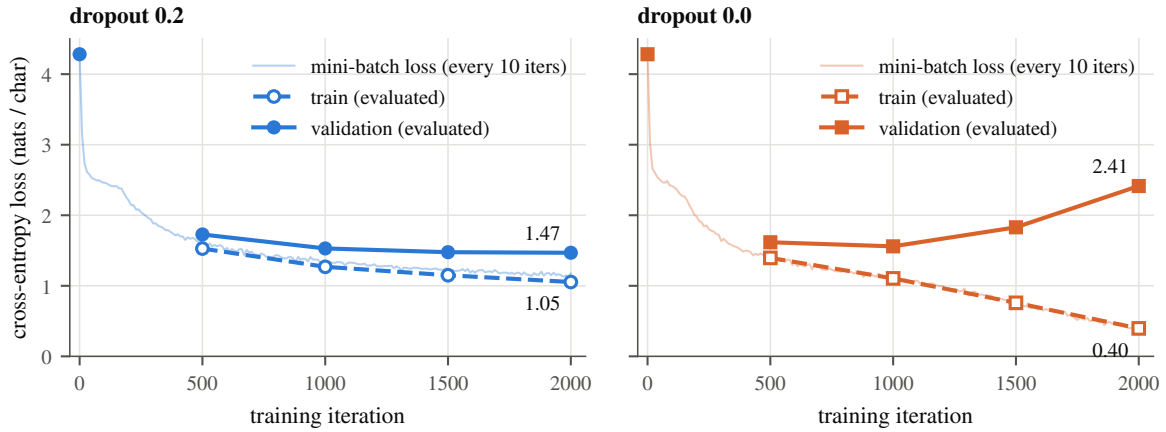


Figure 1: Training curves for the two runs on shared axes. Markers are the losses evaluated every 500 iterations (iteration 0 is 4.28 for both splits); the faint line is the mini-batch training loss logged every 10 iterations.

Table 1: Evaluated loss in nats per character. Gap is validation minus train. Bold marks the best validation loss of each run.

	Dropout 0.2			Dropout 0.0		
Iteration	Train	Validation	Gap	Train	Validation	Gap
0	4.287	4.282	−0.01	4.287	4.282	−0.01
500	1.528	1.728	0.20	1.396	1.617	0.22
1000	1.269	1.530	0.26	1.105	**1.559**	0.45
1500	1.150	1.476	0.33	0.759	1.829	1.07
2000	1.054	**1.468**	0.41	0.396	2.414	2.02

Experiment : The baseline uses dropout 0.2, the value in the reference configuration. The second run sets dropout to 0.0 and changes nothing else, including the random seed (1337); the two runs report identical losses at iteration 0, which confirms a shared initialisation. Every 500 iterations the script estimates the loss on each split as the mean cross-entropy over 200 random batches with dropout switched off. All losses are in nats per character. A checkpoint is written only when the validation loss improves, so the saved model for each run is its best one at this evaluation resolution.

Samples : From each saved checkpoint I draw 10 samples of 500 characters with temperature 0.8 and top-k 200, using the same sampling seed. As a simple check on quality I count the share of generated words that also occur among the 11,676 distinct words of the training split, ignoring case and the truncated final word of each sample.

3 Results

Baseline : Both runs start at 4.28, close to the 4.17 nats ($\ln 65$) of a uniform guess over the vocabulary. With dropout 0.2 the validation loss falls to 1.728 by iteration 500 and to 1.468 by iteration 2,000 (Table 1, Figure 1), which is 2.12 bits per character or a perplexity of 4.34. The improvement is slowing: over the last 500 iterations the validation loss drops by only 0.008 while the training loss drops by 0.096. The generalisation gap widens steadily from 0.20 to 0.41, so the baseline has begun to overfit but its validation loss has not yet turned upwards.

Removing dropout : Without dropout the model fits the training data faster. At iteration 500 it is ahead on both splits, with a validation loss 0.11 lower than the baseline. The advantage

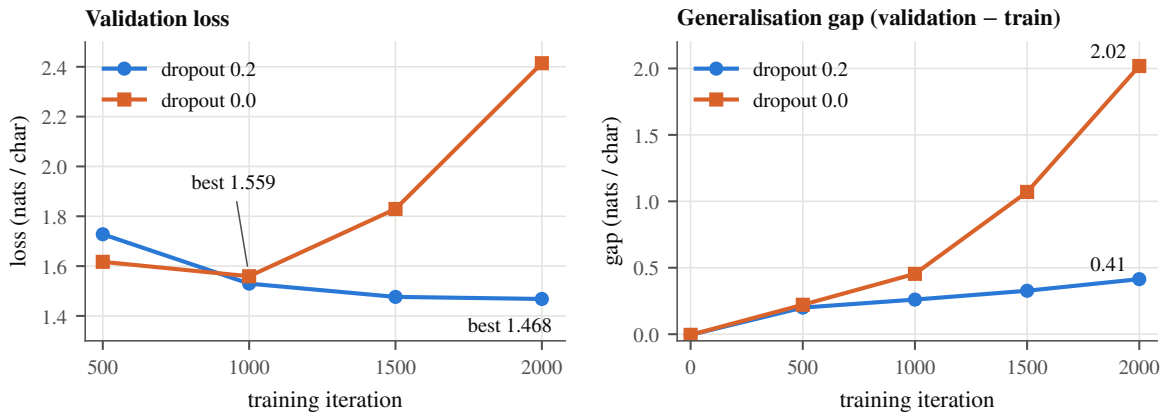


Figure 2: Left: validation loss of the two runs from iteration 500. Right: generalisation gap. The runs are close up to iteration 1,000 and separate afterwards.

is gone by iteration 1,000, where it records its best validation loss of 1.559 against 1.530 for the baseline. After that the two runs diverge (Figure 2). The training loss keeps falling almost linearly to 0.396, while the validation loss rises to 1.829 and then 2.414, worse than its value at iteration 500 and equivalent to a perplexity of 11.2. The final gap of 2.02 nats is 4.9 times the baseline gap. Comparing the best checkpoint of each run, dropout improves validation loss by 0.091 nats (1.468 against 1.559); comparing the runs at iteration 2,000, the difference is 0.95 nats.

The mini-batch trace in Figure 1 gives a second view of the same effect. That loss is computed with dropout active, so in the baseline it sits above the evaluated training loss (1.12 to 1.17 over the last four logged values, against 1.05). In the run without dropout the two coincide (0.39 to 0.42 against 0.40): nothing handicaps the network during training, and it uses its full capacity to fit text it has seen about 33 times over.

Samples : Because checkpoints are saved only on improvement, the samples come from iteration 2,000 for the baseline and iteration 1,000 for the run without dropout. Both models reproduce the format of a play, with speaker names in capitals, line breaks in verse and mostly plausible spelling, and neither produces coherent meaning. The word check separates them: 933 of 961 generated words (97.1%) occur in the training split for the baseline, against 880 of 938 (93.8%) without dropout. The second model produces roughly twice as many words that are absent from the training text (58 against 28), most of them invented forms such as *blushion*, *betwered* and *accunning*. The excerpts below start from the same sampling seed, which is why they open identically.

Dropout 0.2 (iteration 2,000)

```
MENENIUS:
He is battle, for the rogues of a grave instruments
From their envy; the bear-harded betwixt them; who at
The time that was begon at himself:  he was soldier
```

Dropout 0.0 (iteration 1,000)

```
MENENIUS:
He is the content:  stay is before him:
The lords are pleased, and when he now his noble was
our required, and then the hand be cruel and dew
```

4 Limitations

One seed per setting - Each configuration was run once, so there are no error bars. The divergence after iteration 1,000 is large enough that seed variation is unlikely to reverse it, but

the 0.091 difference between best checkpoints, and the lead of the no-dropout run at iteration 500, could each be partly noise.

Coarse evaluation - Loss is measured every 500 iterations. The true minimum of the run without dropout lies somewhere between iterations 500 and 1,500 and may be lower than 1.559, so the best-checkpoint comparison is approximate. Evaluating every 100 iterations would locate it.

Truncated schedule - Training stopped at 2,000 of the 5,000 configured iterations with the learning rate at 70% of its peak. The baseline had not converged, and these runs cannot show how low its validation loss would go or whether it too would eventually rise.

Only two settings - Only dropout 0.0 and 0.2 were tested. Intermediate values, and the interaction with weight decay or model size, are untested, so the results show that some dropout helps here, not that 0.2 is the best value.

Validation data used twice - The validation split both selects the checkpoint and reports its loss, which makes the best-checkpoint figures slightly optimistic. It is also a single contiguous block from the end of the file and may differ in content from the training text.

Narrow evaluation and scope - Sample quality is judged from 10 short samples per model at one temperature, and the word check measures spelling only, not grammar or sense. The two sampled checkpoints also differ in training time. Finally, the conclusion concerns a small model on about 1M characters of one author; it should not be assumed to hold for large corpora, where each example is seen far less often.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

[2] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[3] A. Karpathy. nanoGPT. GitHub repository, 2022. `https://github.com/karpathy/nanoGPT`

[4] A. Karpathy. The unreasonable effectiveness of recurrent neural networks. Blog post, 2015. `https://karpathy.github.io/2015/05/21/rnn-effectiveness/`

[5] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.